Repaso del día 1

16 preguntas sobre el origen del dinero, la banca, la crisis de 2008 y el nacimiento de Bitcoin

Las 16 preguntas — Repaso día 1

1. ¿Cuál fue la principal limitación del trueque que llevó a la aparición del dinero?

2. ¿En qué se diferencia el dinero commodity del dinero convertible?

3. ¿Qué es el dinero fiduciario y cuándo se estableció globalmente?

4. ¿Por qué el dinero fiduciario tiene valor si no está respaldado por un bien físico?

5. ¿Qué es la reserva fraccionaria y cómo nació?

6. Define riesgo de contrapartida y pon dos ejemplos cotidianos.

7. ¿En qué consiste el settlement y por qué es el momento clave de una transacción?

8. ¿Por qué el settlement es complejo y costoso en las transacciones interbancarias actuales?

9. ¿Cuál es la diferencia clave entre cómo liquida Bizum y cómo liquida PayPal?

10. ¿Qué ratio aproximado puede aplicar un banco con la reserva fraccionaria y para qué debe usarlo?

11. ¿Qué falló en la crisis de 2008 desde la perspectiva del riesgo de contrapartida?

12. ¿Qué papel jugaron los bancos centrales antes, durante y después de la crisis de 2008?

13. ¿Qué propuso Satoshi Nakamoto en octubre de 2008 y por qué fue una respuesta directa a la crisis?

14. ¿Qué simboliza la frase 'The Times 03/Jan/2009 Chancellor on brink of second bailout for banks' del genesis block?

15. Enumera los cuatro problemas técnicos que blockchain resuelve para hacer posible el dinero digital sin intermediarios.

16. ¿Qué es el problema del doble gasto y por qué hace que el dinero digital sea fundamentalmente distinto de los archivos?

PREGUNTA 1 / 16

Trueque

Pregunta

¿Cuál fue la principal limitación del trueque que llevó a la aparición del dinero?

Respuesta razonada

El trueque exigía coincidencia de necesidades: yo debía querer lo que tú tenías y, simultáneamente, tú lo que yo tenía. Eso lo hacía inviable a escala y, además, no permitía acumular valor en el tiempo. Para resolverlo aparecieron los primeros dineros commodity (sal, conchas, metales): bienes universalmente aceptados que servían a la vez como medio de intercambio y como reserva de valor.

PREGUNTA 2 / 16

Commodity vs Convertible

Pregunta

¿En qué se diferencia el dinero commodity del dinero convertible?

Respuesta razonada

El dinero commodity es el propio bien con valor intrínseco (sal en la Roma temprana, conchas de caurí, oro): vale por sí mismo. El dinero convertible es un papel o moneda respaldada por una reserva tangible, típicamente oro: tú llevas el billete pero el emisor garantiza canjearlo por oro a una tasa fija. Apareció con los recibos de orfebres en Europa (S. XVII) y se formalizó con el patrón oro del S. XIX.

PREGUNTA 3 / 16

Dinero fiduciario

Pregunta

¿Qué es el dinero fiduciario y cuándo se estableció globalmente?

Respuesta razonada

Es dinero sin respaldo físico cuyo valor depende exclusivamente de la confianza en el emisor (típicamente un Estado). Lo inició China en el S. XIII, pero se estableció globalmente en 1971, cuando Nixon suspendió la convertibilidad del dólar en oro y rompió el último anclaje del sistema monetario internacional al patrón oro. Desde entonces todas las monedas del mundo son fiduciarias.

PREGUNTA 4 / 16

Por qué el dinero tiene valor

Pregunta

¿Por qué el dinero fiduciario tiene valor si no está respaldado por un bien físico?

Respuesta razonada

Por la combinación de cinco factores. Confianza colectiva: todos lo aceptamos porque sabemos que mañana otro lo aceptará. Imposición legal: los Estados lo declaran de curso forzoso para impuestos y transacciones. Escasez controlada: los bancos centrales limitan su emisión. Utilidad práctica: resuelve los problemas del trueque. Y expectativa de estabilidad: la gente lo usa porque espera que mantenga valor en el tiempo.

PREGUNTA 5 / 16

Reserva fraccionaria

Pregunta

¿Qué es la reserva fraccionaria y cómo nació?

Respuesta razonada

En el S. XVII los orfebres europeos observaron que los depositantes raramente retiraban todo su oro a la vez. Empezaron a emitir más recibos que oro tenían en custodia y a prestar el sobrante con interés, creando dinero que no estaba realmente en sus arcas. Multiplica el dinero en circulación pero introduce el riesgo de corridas bancarias. Se institucionalizó con el Banco de Ámsterdam (1609) y el Banco de Inglaterra (1694).

PREGUNTA 6 / 16

Riesgo de contrapartida

Pregunta

Define riesgo de contrapartida y pon dos ejemplos cotidianos.

Respuesta razonada

Es el riesgo de que la otra parte de una transacción no cumpla con su obligación. Lo asumimos a diario sin pensarlo: una hipoteca (¿pagará el cliente al banco?), una compra online con pago por adelantado (¿enviará el vendedor el producto?), un depósito bancario (¿devolverá el banco mi dinero si quiebra?). De esta idea sale la incómoda revelación de que un euro en el bolsillo, en el banco y en una stablecoin no son el mismo euro.

PREGUNTA 7 / 16

Settlement

Pregunta

¿En qué consiste el settlement y por qué es el momento clave de una transacción?

Respuesta razonada

El settlement o liquidación es la entrega final y confirmada de un activo. Marca el punto donde la transacción se cierra de verdad y desaparece el riesgo pendiente. En el pasado implicaba intercambio físico de oro o efectivo; hoy se hace con dinero del banco central (reservas o efectivo). Hasta que no hay settlement, el pago no es definitivo: existe la promesa, pero no la entrega.

PREGUNTA 8 / 16

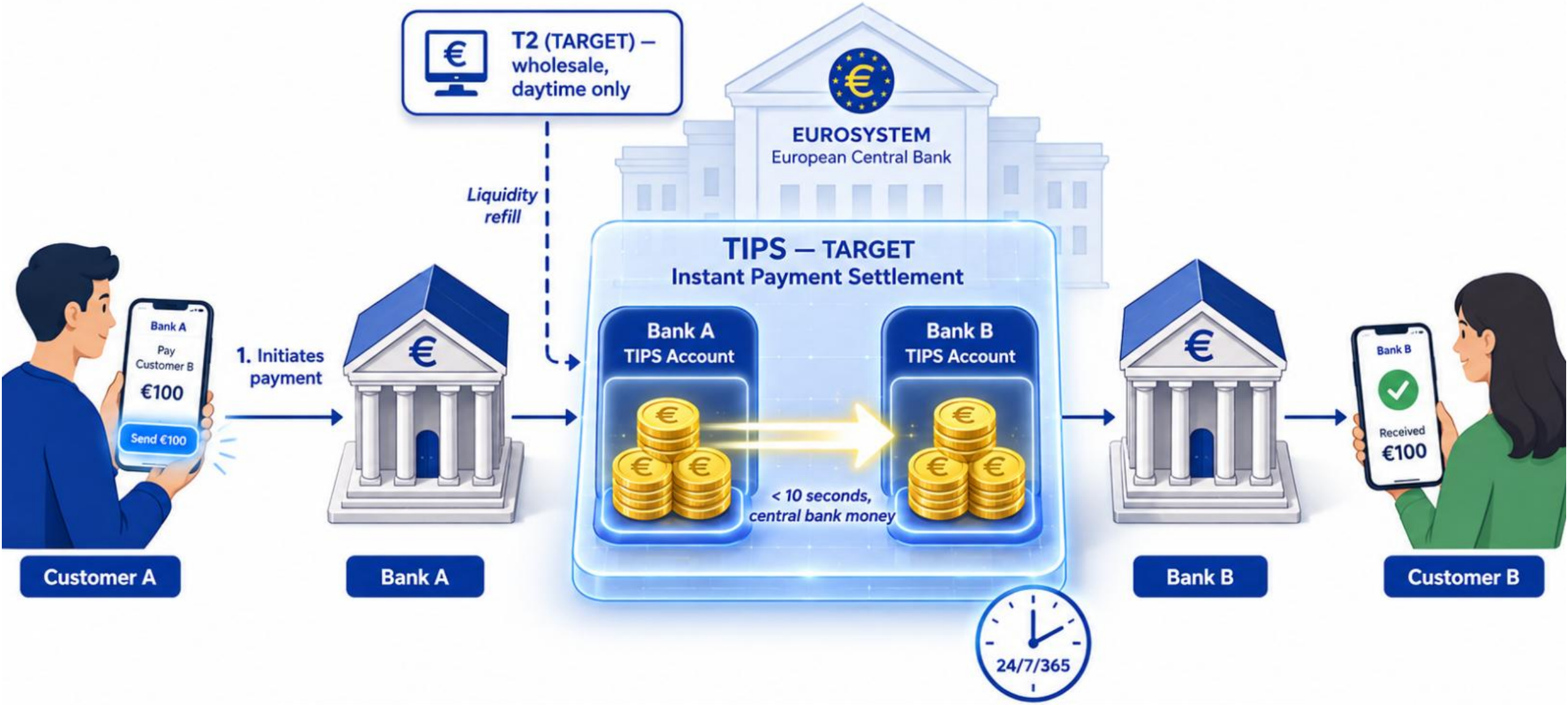
Settlement bancario complejo

Pregunta

¿Por qué el settlement es complejo y costoso en las transacciones interbancarias actuales?

Respuesta razonada

Porque autorización, clearing y liquidación ocurren en procesos y entidades distintas, los bancos no confían plenamente entre sí, las infraestructuras tienen calendarios separados (cierre de bolsa, festivos, husos horarios), hay múltiples intermediarios en cross-border (corresponsales) y comisiones acumuladas etapa tras etapa. Resultado: tiempos poco predecibles, poca transparencia y un coste operacional elevado.



- Instant payments around the clock, every day of the year
- TIPS settles in central bank money (final and irrevocable)
- Secure, efficient and resilient infrastructure
- Part of the Eurosystem supporting European payments

PREGUNTA 9 / 16

Bizum vs PayPal

Pregunta

¿Cuál es la diferencia clave entre cómo liquida Bizum y cómo liquida PayPal?

Respuesta razonada

Bizum liquida en dinero de banco central vía SEPA Instant: el dinero pasa de un banco al otro y la liquidación final es inmediata. PayPal mueve saldo internamente entre sus propias cuentas (no entre bancos) y solo después ejecuta una transferencia bancaria tradicional para que el usuario retire el dinero. Bizum es transmisión real de dinero bancario; PayPal es un libro contable interno con liquidación bancaria diferida.

PREGUNTA 10 / 16

Multiplicación del dinero

Pregunta

¿Qué ratio aproximado puede aplicar un banco con la reserva fraccionaria y para qué debe usarlo?

Respuesta razonada

El ratio es aproximadamente 10 a 1: por cada euro depositado, el banco puede crear hasta diez en forma de préstamos. La condición regulatoria es que ese dinero creado se dedique a préstamos (no a consumo del propio banco). El argumento es que los bancos seleccionan los proyectos más prometedores y multiplican así el dinero del Banco Central; el problema es que dependen de su capacidad para evaluar correctamente el riesgo.

PREGUNTA 11 / 16

Crisis 2008

Pregunta

¿Qué falló en la crisis de 2008 desde la perspectiva del riesgo de contrapartida?

Respuesta razonada

Los bancos prestaron a clientes de baja solvencia (hipotecas subprime), empaquetaron esas hipotecas en productos financieros que parecían seguros y los vendieron entre ellos. Cuando los impagos crecieron, se rompió la confianza en la solvencia bancaria: los bancos dejaron de prestarse entre sí porque no confiaban en cobrar. Eso congeló el settlement interbancario, derivó en corridas bancarias modernas y forzó rescates masivos.

PREGUNTA 12 / 16

Bancos centrales en 2008

Pregunta

¿Qué papel jugaron los bancos centrales antes, durante y después de la crisis de 2008?

Respuesta razonada

Antes: permitieron una expansión crediticia excesiva y una regulación laxa del riesgo. Durante: actuaron como prestamistas de última instancia, inyectando liquidez masiva y rescatando bancos para evitar el colapso. Después: reforzaron la regulación (mayores requisitos de capital, supervisión más estricta) y mantuvieron políticas monetarias expansivas (tipos bajos, compras de activos). Quedó claro que el sistema dependía de la confianza en el dinero bancario, no solo en el del banco central.

Whitepaper de Bitcoin

Pregunta

¿Qué propuso Satoshi Nakamoto en octubre de 2008 y por qué fue una respuesta directa a la crisis?

Respuesta razonada

Propuso un sistema de dinero digital descentralizado y sin necesidad de bancos, basado en criptografía y consenso distribuido. La crisis había hecho visible que la confianza en intermediarios financieros era frágil; el whitepaper de Bitcoin planteaba justo eliminar ese punto único de fallo. No es solo un activo financiero: es una respuesta política y técnica a la fragilidad del dinero bancario evidenciada en 2008.

PREGUNTA 14 / 16

Genesis block

Pregunta

¿Qué simboliza la frase 'The Times 03/Jan/2009 Chancellor on brink of second bailout for banks' del genesis block?

Respuesta razonada

Es la portada del Times del 3 de enero de 2009, día en que Satoshi minó el primer bloque, y hablaba del segundo rescate bancario que el Canciller británico estaba preparando. Cumple dos funciones: como sello temporal criptográfico (timestamp que demuestra que ese bloque no se minó antes), y como declaración política del proyecto: Bitcoin nace como alternativa a un sistema que rescata bancos con dinero público.

PREGUNTA 15 / 16

Cuatro problemas de blockchain

Pregunta

Enumera los cuatro problemas técnicos que blockchain resuelve para hacer posible el dinero digital sin intermediarios.

Respuesta razonada

Doble gasto: evitar que la misma moneda se use dos veces sin un árbitro central. Orden de las transacciones: en una red descentralizada los nodos ven mensajes en distinto orden y hace falta un mecanismo compartido de secuencia. Integridad de los registros: una vez acordado el orden, garantizar que nadie pueda alterar el pasado (encadenamiento por hash). Y confianza sin intermediario: la propia red verifica colectivamente cada transacción, sustituyendo al banco central como árbitro.

PREGUNTA 16 / 16

Doble gasto

Pregunta

¿Qué es el problema del doble gasto y por qué hace que el dinero digital sea fundamentalmente distinto de los archivos?

Respuesta razonada

Un fichero digital se puede copiar trivialmente — copiar/pegar deja dos copias idénticas — y eso es irrelevante para una foto, pero con dinero sería fraude: gastarías la misma moneda dos veces. Antes de Bitcoin, toda forma de dinero digital requería un tercero central (banco, PayPal) que llevara la cuenta de quién tiene qué y validara cada operación. La innovación de blockchain es resolver ese problema sin árbitro: cada transacción se valida colectivamente y queda registrada de forma inmutable.

Hash

¿Qué es un Hash?

1 Función de una sola dirección

Un hash es una función de **una sola dirección** que convierte datos de entrada en una salida única de **tamaño fijo**. Se puede ver como una huella digital de los datos.

Entrada (datos) → Función de hash → Salida (hash)

2 Una sola dirección

Es de una sola dirección porque es sencillo generar la salida, pero dada una salida es **imposible*** calcular los **datos originales** que la generaron.

Salida (hash) → ⛔ → Entrada (datos)

3 Determinismo

La misma entrada siempre produce el **mismo hash**.

Entrada: Hola mundo → 98f7b6e1c3d2...

Entrada: Hola mundo → 98f7b6e1c3d2... ✔ Siempre el mismo resultado

4 Efecto avalancha

Un pequeño **cambio** en la entrada produce un **cambio drástico** y aparentemente aleatorio en la salida.

Entrada original: Hola mundo → Hash resultante: 98f7b6e1c3d2a4f5...

Entrada con 1 cambio: Hola mundo! → 3a4c7d91b8e0f2a1... ! Salida completamente diferente

5 Características clave (resumen)

Rápido
Crear el hash dada una entrada es rápido.

Sin colisiones
Es imposible* que dos entradas diferentes generen la misma salida.

Determinista
La misma entrada siempre produce el mismo hash.

Efecto avalancha
Un pequeño cambio en la entrada genera un cambio grande e impredecible en la salida.

Tamaño fijo
La salida (hash) siempre tiene el mismo tamaño, independientemente de la entrada.

Recordamos: MyHash con P = 7

Definición: MyHash(x) = x módulo P, con P primo

Ejemplo: P = 7

MyHash(21) = 0 (porque 21 / 7 = 3, resto 0)

MyHash(33) = 5 (porque 33 / 7 = 4, resto 5)

MyHash(100) = 2 (porque 100 / 7 = 14, resto 2)

Cumple aparentemente las propiedades:

- De una sola dirección: dado el resto no recuperas el input
- Rápido: una división
- Salida fija: siempre un número entre 0 y 6

Pero, ¿cumple lo de 'sin colisiones'? Vamos a comprobarlo.

ACTIVIDAD

Reto: encuentra 10 colisiones de MyHash con P = 7

Tienes 1 minuto. Escribe 10 pares (a, b) con a ≠ b tales que MyHash(a) = MyHash(b).

Pista: si MyHash(x) = r, entonces MyHash(x + 7) = r, MyHash(x + 14) = r, etc.

Ejemplo gratis: MyHash(3) = MyHash(10) = MyHash(17) = MyHash(24) = 3.

Al acabar, comenta con tu compañero:

- ¿En cuánto tiempo encontraste las 10?
- Compáralo con el '1 entre 10^{72}' que dice la slide de SHA-256.

PIGEONHOLE

Por qué MyHash siempre va a fallar

Con P = 7 hay solo 7 outputs posibles: {0, 1, 2, 3, 4, 5, 6}.

Pero los inputs son infinitos.

Principio del palomar: si tienes 8 palomas y 7 nidos, dos comparten nido.

Aplicado aquí: con 8 inputs distintos ya tienes garantizada al menos una colisión.

Has demostrado a mano lo que para SHA-256 nos costaría más que la edad del universo.

- MyHash(P = 7): colisión en segundos.
- SHA-256: colisión nunca observada en 24 años.

Pregunta natural del alumno: ¿y si hacemos P enorme?

El siguiente intento: hagamos P enorme

Idea: tomar P = un primo de 128 bits (≈ 10^{38} valores posibles).

El espacio de outputs ahora es gigantesco. Las colisiones por casualidad dejan de ser triviales.

Pero MyHash sigue teniendo dos problemas graves que SHA-256 sí resuelve:

1. Predictivo: si conoces el output r y P, puedes generar infinitos inputs (r, r + P, r + 2P, ...)

   Un atacante elige el input que le conviene para que cuadre con un hash dado.

2. Sin avalanche: cambiar el input en 1 cambia el output en 1.

   Inputs parecidos producen outputs parecidos. Patrón visible.

MyHash es matemáticamente lineal. Eso es exactamente lo que NO queremos en un hash criptográfico.

SOLUCIÓN

Por qué SHA-256 sí funciona

Outputs: $2^{256} \approx 10^{77}$ valores. Más que átomos en el universo observable.

Avalanche real: cambiar 1 bit del input cambia ≈ 50% de los bits del hash.

No hay correlación visible entre entradas parecidas.

No invertible y no predictivo: nadie sabe diseñar un input para un output concreto.

Las operaciones internas mezclan, rotan, hacen XOR y desplazan tantas veces que la trazabilidad inversa se pierde.

Resistencia comprobada: publicado en 2001 por la NSA, no hay colisión conocida en 24 años.

Los matemáticos llevan medio siglo intentando romper hashes. SHA-256 sigue en pie.

Moraleja: 'sin colisiones' no es un truco mágico, es ingeniería matemática refinada durante décadas.

Algoritmos de hash: cuáles siguen vivos, cuáles no

Algoritmo	Bits	Estado	Dónde se usa
MD5	128	ROTO (2004)	Solo checksums no críticos. Nunca en cripto.
SHA-1	160	ROTO (2017)	Git (migrando), TLS antiguo. En retirada.
SHA-256	256	Vigente	Bitcoin, Hyperledger Fabric, TLS, certificados.
Keccak-256	256	Vigente	Ethereum, direcciones, Merkle trees.
SHA-3 (familia)	224–512	Vigente	Estándar NIST 2015. Sustituto de SHA-2.
BLAKE3	variable	Vigente	Sistemas modernos que priorizan velocidad.

EJEMPLO REAL

shattered.io — la colisión real de SHA-1 (2017)

Google y CWI publicaron dos PDFs distintos con exactamente el mismo SHA-1.

Coste del ataque: ≈ 110 GPUs durante 1 año.

Antes de 2017 se decía que era inviable. En 2017 se demostró que ya no lo era.

Conclusión: la criptografía caduca.

- MD5 cayó en 2004.
- SHA-1 cayó en 2017.
- SHA-256 sigue en pie en 2026, pero no para siempre.

Por eso los protocolos serios incluyen 'crypto agility':

poder cambiar de algoritmo sin reconstruir todo el sistema.

Por eso Bitcoin usa SHA-256 (conservador) y por eso Fabric expone el algoritmo en la configuración.

EL HASH EN BLOCKCHAIN: EL PEGAMENTO QUE LO HACE POSIBLE

El hash convierte datos en una "huella digital" única e irreversible.
En Blockchain, esta huella se usa para dar integridad, seguridad y unicidad.

¿PARA QUÉ SE USA EL HASH EN BLOCKCHAIN?

1 INTEGRIDAD DE DATOS EN EL ENCADENADO DE BLOQUES

Cada bloque incluye el hash del bloque anterior.
Si alguien altera un dato, cambia su hash
y se rompe toda la cadena.

Bloque n-1	Bloque n	Bloque n+1
Datos	Datos	Datos
Hash anterior: 7a3f...9c21	Hash anterior: d4e1...8b7a	Hash anterior: a1b2...4c3d
Hash del bloque: d4e1...8b7a	Hash del bloque: a1b2...4c3d	Hash del bloque: f6e7...2d9b

...

Si se modifica un dato en un bloque, su hash cambia
y todos los bloques siguientes dejan de ser válidos.

2 ALGORITMOS DE CONSENSO

Muchos consensos (como Proof of Work)
se basan en encontrar un hash que cumpla
una condición (ej: que empiece por ceros).

Buscar un nonce
tal que:
SHA-256(datos || nonce)
empiece por **00000**

2f8c...a1b2 ✗
91aa...fe01 ✗
00000a4f...7c3e ✓

El primero en encontrarlo añade
el bloque a la cadena.

3 CREACIÓN DE IDENTIFICADORES ÚNICOS

El hash permite crear identificadores únicos
de datos, transacciones, direcciones o contratos.

Transacción	b3c6...9e7f
Bloque	a1b2...4c3d
Dirección	e4d8...1a9b
Smart Contract	9f7e...0d2c

El hash convierte cualquier dato en una huella única.
Pequeño cambio en los datos → hash completamente diferente.

MÚLTIPLES FUNCIONES MATEMÁTICAS DE HASHING

CRC	MD	SHA-1	SHA-2	SHA-3	Keccak	SHAKE	KMAC	BLAKE
Detección de errores en datos. No criptográfico.	Familia antigua de hashes criptográficos (MD2, MD4, MD5).	Muy usado en el pasado. Hoy en día se considera débil.	Familia segura y muy extendida (SHA-224, SHA-256, SHA-512...).	Ganador del concurso NIST. Diseñado desde cero.	Base de SHA-3. Estructura de esponja criptográfica.	Funciones de salida extensible (XOF).	Hash basado en clave, parte del estándar SP 800-185.	Rápido, seguro y eficiente. Usado en muchas aplicaciones.

Todas producen una salida de tamaño fijo (o configurable)
a partir de datos de cualquier tamaño.

¿QUÉ USA CADA BLOCKCHAIN?

BITCOIN
Usa
SHA-256
Cada bloque y transacción
se identifica con SHA-256.

ETHEREUM
Usa
KECCAK-256
Ethereum utiliza Keccak-256
(base de SHA-3) para sus hashes.

SIN HASH, NO HAY BLOCKCHAIN. El hash garantiza que cualquier intento de alterar datos sea detectable,
que todos estén de acuerdo en el estado de la red y que cada elemento sea único e identificable.

Merkle Tree: el resumen criptográfico de un conjunto

Estructura: árbol binario donde cada hoja es el hash de un dato.

Cada nodo interno es el hash de la concatenación de sus dos hijos.

La raíz (Merkle root) resume todo el árbol en un solo hash de 32 bytes.

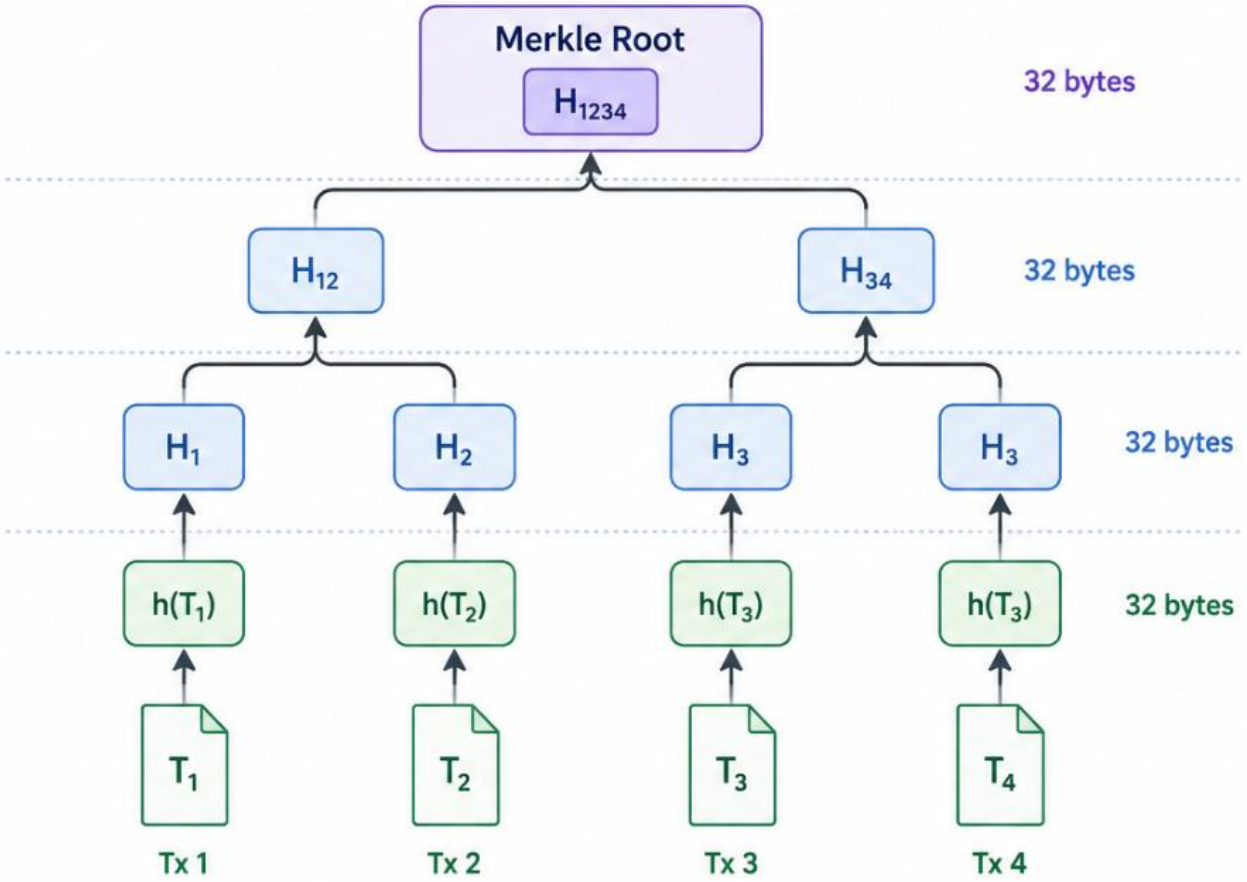
Ejemplo Bitcoin: cada bloque incluye la Merkle root de todas sus transacciones.

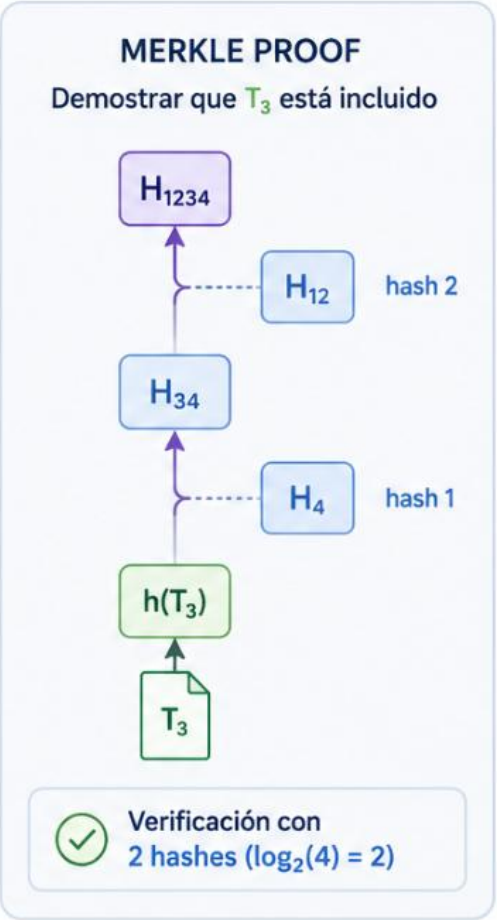
- Cambia una sola transacción → cambia la raíz → bloque inválido.
- Una huella diminuta autentica miles de transacciones.

Merkle proof: demostrar que un dato está en el conjunto enviando solo $\log_2(N)$ hashes.

- 1024 transacciones → 10 hashes ≈ 320 bytes para probar inclusión.
- Es lo que permite a los nodos ligeros (SPV) verificar sin descargar la cadena completa.

MERKLE TREE





T_n Transacción / Dato

$h(T_n)$ Hash de la transacción

H_n Nodo interno (hash)

H_{1234} Merkle Root (resumen de todo)

Un pequeño cambio en cualquier T_n cambia la raíz H_{1234}

Blockchain - Hash

https://github.com/jjnieto/Curso-HLFabric/

Clave Pública/Privada

Blockchain – Claves pública/privada

Son dos números matemáticamente ligados, de forma que de la calve privada se puede calcular la pública fácilmente pero no a la inversa. Cumplen estas dos importantes propiedades:

1. La clave privada es secreta y permite descifrar lo que fue cifrado con la.

2. Con tu **clave privada** puedes, dado un mensaje cualquiera, generar un **código único** que solo esa clave privada puede producir y se verifica con la clave pública. Ese código es una firma digital.

Las dos operaciones de la criptografía asimétrica

	Cifrar / Descifrar	Firmar / Verificar
Quién hace la 'ida'	Cualquiera con tu pública	Solo tú con tu privada
Quién hace la 'vuelta'	Solo tú con tu privada	Cualquiera con tu pública
Para qué sirve	Confidencialidad	Autenticidad + integridad
Analogía	Candado abierto que regalas	Sello personal + verificación
Cuándo se usa	Mensajes secretos, sesiones cifradas	Transacciones, identidad, contratos

ANALOGÍA — CIFRAR

El candado abierto

Imagina un candado del que solo tú tienes la llave única (clave privada).

Fabricas miles de copias de ese candado abierto y los repartes (clave pública).

Cualquiera puede coger un candado tuyo, meter su mensaje en una caja y cerrarla.

Una vez cerrada, ni quien la cerró puede volver a abrirla.

Solo tú, con tu llave, puedes hacerlo.

Resultado: confidencialidad.

Aunque alguien intercepte la caja por el camino, ve un cofre cerrado.

Equivalencia criptográfica:

- Candado abierto → clave pública (se reparte sin riesgo).
- Llave única → clave privada (jamás sale de tu poder).
- Caja cerrada → mensaje cifrado.

ANALOGÍA — FIRMAR

El sello personal

Imagina un sello único que solo tú posees (clave privada).

Lo estampas en cualquier documento que quieras 'autenticar como tuyo'.

Repartes a todo el mundo una hoja con la imagen exacta de tu sello (clave pública).

Cualquiera puede comparar el sello del documento con la imagen de referencia.

Resultado: dos garantías a la vez.

- Autenticidad: si el sello coincide, el documento lo firmaste tú.
- Integridad: si alguien cambia una sola coma del documento, el sello deja de coincidir.

Equivalencia criptográfica:

- Sello físico → clave privada (firma única, irrepetible).
- Imagen de referencia → clave pública (verificación abierta a todos).
- Documento estampado → mensaje firmado.

CRIPTOGRAFÍA ASIMÉTRICA

Un par de claves: **pública** (para todos) y **privada** (solo para ti)

1) CONFIDENCIALIDAD (CIFRADO)

Clave privada → Clave pública → Cualquiera

1. Mensaje → 2. Se cifra con clave pública → 3. Solo tu clave privada lo abre

EQUIVALENCIA

- Candado abierto
- Llave única
- Caja cerrada

2) AUTENTICIDAD E INTEGRIDAD (FIRMA)

Clave privada → Clave pública → Cualquiera

1. Documento → 2. Se firma con clave privada → 3. Cualquiera verifica

EQUIVALENCIA

- Sello único
- Imagen de referencia
- Documento firmado

Pública para compartir, privada para proteger.

ATENCIÓN

¿Y si pierdo mi clave privada?

Si la pierdes: no hay forma matemática de recuperarla a partir de la pública.

Si la filtras: quien la tenga puede firmar como tú y mover tu dinero. No hay forma de revertirlo.

Diferencia radical con la banca tradicional:

- Banco: pierdes la tarjeta → llamas, te emiten otra. Tu dinero sigue ahí.
- Blockchain: pierdes la privada → perdiste el dinero. Punto.

Hay miles de millones de dólares en BTC literalmente bloqueados así.

El más conocido: el disco duro de James Howells, tirado a la basura con 8.000 BTC.

Por eso existen mecanismos para reducir el riesgo:

- Hardware wallets (Ledger, Trezor): la privada vive en hardware sellado.
- Frases semilla (BIP-39): 12 o 24 palabras que regeneran la privada.
- Custodios centralizados (exchanges): te quitan el problema, pero te fías de ellos.
- Multi-sig: necesitas N de M firmas. Si pierdes una, las otras te salvan.

Blockchain – Claves pública/privada

Pregunta:

¿Por qué no puedo usar simplemente una clave privada elegida por mi al azar y un algoritmo de hash para calcular mi clave pública y que encripten con ella? Después de todo, la clave pública se calcularía fácilmente desde la privada.

RESPUESTA

Lo que hace falta es una función trampa, no un hash

Para que el cifrado asimétrico funcione, la operación tiene que cumplir dos cosas:

1. Ser fácil de aplicar con la pública (cifrar es rápido para todos).
2. Ser fácil de deshacer SOLO para quien tenga la privada (descifrar exige el secreto).

A esto se le llama función trampa, o one-way trapdoor function:

- One-way en general: nadie puede invertirla.
- Pero con la clave privada hay una 'trampa' que la hace fácilmente reversible.

Ejemplos reales:

- RSA: multiplicar dos primos enormes es trivial; factorizar el resultado es inviable.

  La privada incluye los dos primos → puedes descomponerlo → puedes descifrar.
- ECC (curvas elípticas): multiplicar un punto por un escalar es trivial;

  recuperar el escalar conocido el punto resultante es inviable (logaritmo discreto).

Un hash no tiene trampa. Es one-way puro: ni tú ni nadie puede invertirlo.

Por eso la respuesta es no: el hash sirve para huellas de integridad, no para cifrar.

Para cifrado asimétrico necesitas matemática construida con trampa secreta.

Blockchain - Hash

https://github.com/jjnieto/Curso-HLFabric/

Blockchain – Claves pública/privada

¿Para qué se usan las claves pública/privada en blockchain?

De la clave pública a la dirección Bitcoin

1. Tu clave pública es un punto sobre la curva secp256k1: ≈ 65 bytes en hex.
2. Aplica SHA-256 → 32 bytes.
3. Aplica RIPEMD-160 al resultado → 20 bytes.
4. Añade un byte de versión delante (0x00 para mainnet) + checksum (4 bytes del doble SHA-256).
5. Codifica todo en Base58Check.

Resultado: 1A1zP1eP5QGefi2DMPTfTL5SLmv7DivfNa (la primera dirección de Bitcoin, de Satoshi).

Por qué cada paso:

- Doble hash (SHA-256 + RIPEMD-160): redundancia. Si alguien rompe SHA-256, el segundo aún protege.
- Base58 (no Base64): elimina caracteres confundibles (0, O, I, l) y signos.
- Checksum integrado: detecta una errata de un solo carácter al copiar la dirección.

Bitcoin vs Ethereum: dos formas de derivar la dirección

Bitcoin

Curva: secp256k1.

Pasos: Pública → SHA-256 → RIPEMD-160 → versión + checksum → Base58Check.

Formato: 1A1zP1eP5QGefi2DMPTfTL5SLmv7DivfNa

Doble hash → robustez frente a futuras roturas.

Checksum incluido en la propia dirección.

Sin caracteres ambiguos (0, O, I, l fuera).

Ethereum

Curva: secp256k1 (la misma que Bitcoin).

Pasos: Pública → Keccak-256 → últimos 20 bytes → prefijo '0x'.

Formato: 0xd35d2af6f8a3f0c8b3a9e1c75e9bb2c0bf7c6f8a

Un solo hash → más simple.

Sin checksum nativo (EIP-55 lo añadió después con may/min).

Hex puro, optimizado para máquinas.

¿PARA QUÉ SE USAN LAS CLAVES PÚBLICA/PRIVADA EN BLOCKCHAIN?

Para crear las wallets de los clientes y firmar transacciones con la clave privada asociada a ese wallet.
Es una especie de identidad digital pseudónima.

PSEUDÓNIMO
No revelas tu identidad real, solo tu dirección.

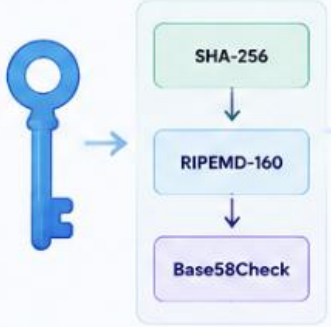
1 GENERA TU PAR DE CLAVES

CLAVE PRIVADA
Secreta
(se guarda en tu wallet)

CLAVE PÚBLICA
Se deriva de la privada

La clave privada nunca se revela.

2 DE LA PÚBLICA SE CALCULA UNA DIRECCIÓN



(Ejemplo Bitcoin)

3 TU DIRECCIÓN: LO QUE COMPARTES PARA RECIBIR FONDOS

La compartes con cualquiera para que te envíen fondos.

4 FIRMAS TRANSACCIONES CON TU CLAVE PRIVADA

Solo tú puedes firmarla con tu clave privada.

5 LA RED VERIFICA CON TU CLAVE PÚBLICA

CUALQUIERA PUEDE VERIFICAR
la firma con tu clave pública.
Tu identidad permanece pseudónima.

PIENSA EN TU WALLET ASÍ:

TU DIRECCIÓN = TU NÚMERO DE CUENTA
(la compartes para recibir)

TU CLAVE PÚBLICA = RESPALDA TU DIRECCIÓN
(verificable por todos)

TU CLAVE PRIVADA = LA ÚNICA FORMA DE MOVER TU DINERO
(mantenla en secreto)

EN RESUMEN

Tus claves te dan control de tus fondos sin revelar quién eres.
Eres dueño de tu dinero y tu identidad en la red es pseudónima.

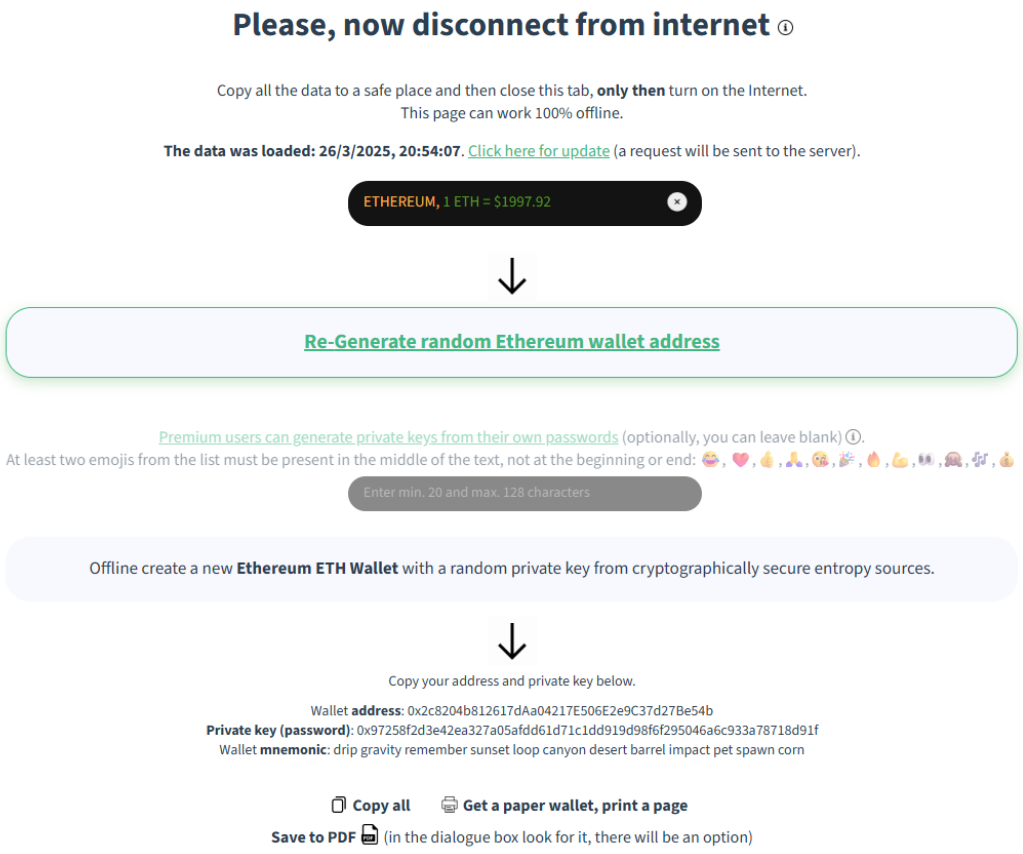
Tú = Dirección
(pero no tu nombre real)

Actividad – Claves pública/privada

https://www.bitaddress.org/

Actividad – Claves pública/privada

https://generate.mitilena.com/en/ethereum/



Redes P2P

El modelo familiar: cliente-servidor

Es el modelo dominante en la web e Internet de consumo.

Hay dos roles fijos y separados:

- Servidor: máquina siempre encendida, dirección conocida, presta el servicio.
- Cliente: máquina del usuario, conecta cuando quiere, consume el servicio.

Ejemplos típicos:

- Web: tu navegador (cliente) → google.com (servidor).
- Email: tu app de correo → servidor SMTP/IMAP de Gmail.
- Banca online: tu móvil → servidor del banco.

Funciona muy bien cuando hay confianza centralizada y una entidad responsable.

Los límites del cliente-servidor

1. Punto único de fallo. Si el servidor cae, el servicio se cae.
   - Caída de AWS = caída de medio Internet ese día.

2. Punto único de control. Quien controle el servidor controla el servicio.
   - Bloqueos, censura, modificaciones unilaterales.

3. Coste a escala. Si el servicio crece, el servidor se sobrecarga.
   - Black Friday: webs caídas por exceso de clientes.

4. Confianza obligatoria. El cliente confía en que el servidor le da la verdad.
   - No tienes forma de verificar que tu saldo bancario es correcto. Crees al banco.

Peer-to-peer: cada nodo es cliente y servidor a la vez

En una red P2P no hay roles fijos. Cada participante (peer):

- Pide recursos cuando los necesita (actúa como cliente).
- Sirve recursos a otros peers (actúa como servidor).

No hay un servidor central. La red emerge de la unión de todos los peers.

Si un peer se cae, los demás siguen funcionando.

Si entran 1.000 peers nuevos, la red se hace más fuerte (cada uno aporta capacidad).

Idea fundamental: el servicio no vive en una máquina, vive en el conjunto.

Cuanta más gente participe, mejor funciona.

Es el modelo opuesto al cliente-servidor: distribuido, sin centro, autoorganizado.

CASO HISTÓRICO

Napster (1999): el primer P2P de masas

Shawn Fanning, 18 años, inventa Napster para compartir MP3 entre usuarios.

En 18 meses pasa de 0 a 80 millones de usuarios. Pánico en la industria musical.

Arquitectura híbrida:

- Los archivos están en los ordenadores de los usuarios (eso sí era P2P).
- Pero un ÍNDICE CENTRAL (servidor de Napster) sabía quién tenía qué.
- Buscabas 'Metallica - One' → el servidor te decía qué peer lo tenía.
- Después tu cliente conectaba directamente con ese peer.

Ese diseño tenía un punto débil enorme: el servidor central de búsqueda.

En 2001 la RIAA demanda. Cierran el servidor → la red entera se cae.

Lección: si hay UNA pieza centralizada, la red entera es vulnerable a ese punto.

Esta es la lección que Bitcoin va a interiorizar 8 años después.

La respuesta: P2P puro (Gnutella, eMule, Kazaa)

Después de Napster, los proyectos eliminan el servidor central de búsqueda.

Todo se hace entre peers, incluyendo el descubrimiento de archivos.

Cómo se busca sin servidor: 'flooding'.

- Tu peer pregunta a sus vecinos: '¿alguien tiene Metallica - One?'
- Cada vecino propaga la pregunta a sus vecinos. Y así sucesivamente.
- Los que tengan el archivo responden directamente al peer original.

Resultado: la red sobrevive aunque caigan miles de nodos. Imposible cerrarla por demanda.

- eMule: > 100 millones de descargas en su pico.
- Gnutella: 1,8 millones de usuarios simultáneos.
- Kazaa: 250 millones de usuarios totales.

Problema del flooding: muy ineficiente. Cada búsqueda genera miles de mensajes en la red.

Funciona pero no escala bien. Hace falta una idea mejor.

EVOLUCIÓN

BitTorrent (2001): partir el archivo y descargar en paralelo

Bram Cohen propone una idea radicalmente eficiente:

- El archivo se parte en muchos trozos (pieces) de ~256 KB cada uno.
- Cada peer descarga trozos distintos de peers distintos en paralelo.
- Cada peer SUBE los trozos que ya tiene mientras descarga los que le faltan.

Quien comparte más, descarga más rápido (incentivo tit-for-tat).

Lo que se forma se llama swarm: el enjambre de peers que comparten un archivo.

BitTorrent es la primera red P2P que combina:

- Eficiencia: descarga paralela de trozos.
- Resiliencia: si caen peers, el swarm sigue.
- Incentivos: el que más comparte, mejor velocidad.

Sigue vivo en 2026. Distribución de Linux, juegos, contenido legal y no tan legal.

P2P frente a cliente-servidor: lo bueno y lo difícil

Ventajas de P2P

Sin punto único de fallo. Caen peers, la red sigue.

Resistente a censura. No hay servidor que apagar.

Escalabilidad por participación. Cada nuevo peer añade capacidad.

Coste compartido. No hace falta una empresa pagando hosting.

Verificable. Puedes comprobar la integridad de lo que recibes (con hashes).

Problemas que hay que resolver

Descubrimiento: ¿cómo encuentro mis primeros peers? (Bootstrap nodes).

NAT y firewalls: la mayoría de peers no son accesibles directamente.

Free riders: peers que solo descargan y no aportan. Necesitan incentivos.

Búsqueda eficiente: flooding no escala, hace falta DHT u otra estructura.

Confianza entre desconocidos: si nadie es responsable, ¿en quién confías?

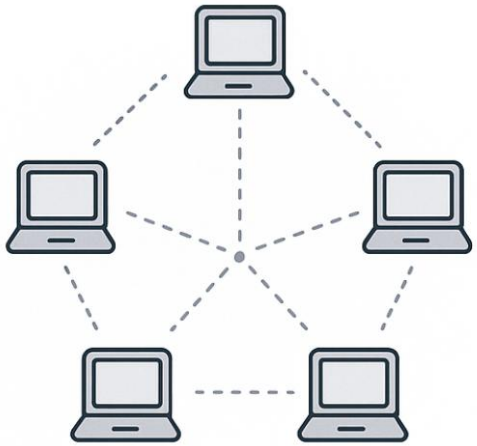
Sybil attacks: un atacante crea miles de identidades falsas para tomar la red.

Blockchain – Red peer-to-peer

1. La red blockchain P2P es un software que se ejecuta en cada nodo

Cualquier ordenador que instale el cliente de la red (Bitcoin Core, Geth, etc.) se convierte en un nodo

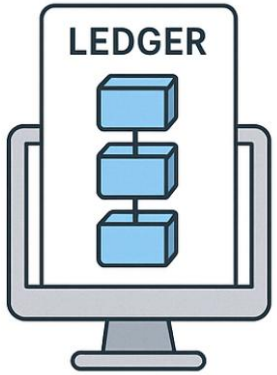
2. Al arrancar, el nodo se conecta a otros nodos



Busca pares conocidos o consultados en una lista inicial.

Establece conexiones como en eMule o BitTorrent

3. Una vez conectado, descarga y almacena el ledger



Cada nodo guarda una copia completa del ledger (cadena de bloques).

Eso asegura redundancia; el historial no depende de un servidor central